Algorithmic Thinking and C++ Basics

Computer Programming 1 · Lecture 2

Siyavushkhon Kholmatov · New Uzbekistan University

Week 2 · Fall 2026

Where we are

Last week
Computers, information, and how a machine stores numbers: bits, bytes, binary, hexadecimal.

Today
How to think about a problem — then how to say it in C++.

Next week
Operators, expressions and control flow in real code.

Thinking before coding

Programming is not typing

	Stage	What you produce
1	Analyse	The *input*, the *output*, and the process between them.
2	Plan	The algorithm — pseudocode, a flowchart, or both.
3	Code	Source code obeying one language's syntax.
4	Test	Runs on data chosen to expose faults.
5	Maintain	Documentation and updates.

When a test fails you return to stage 2, not stage 3.

Two ways a program can be wrong

Syntax error

You broke a rule of the language — a missing semicolon, an unclosed brace.

The compiler stops and names the line. Cheap to fix.

Logic error

Your instructions are legal C++ — and wrong.

It compiles. It runs. It prints 47 when the answer is 74.

What an algorithm is

Definition

A finite, ordered sequence of unambiguous steps that solves a problem or performs a task.

Finite	It must stop. A procedure that loops forever has solved nothing.
Ordered	Swapping two steps may change the result.
Unambiguous	“Add some salt” is not a step; “add 5 grams of salt” is.
Effective	Each step can actually be carried out. “Guess the answer” cannot.

Your turn — which are not algorithm steps?

(a) Set $grade \leftarrow grade + 5$.

(b) Repeat until A+

(c) If $y > 60$, print “many”.

(d) Sort the list somehow.

Your turn — which are not algorithm steps?

(a) Set *grade* $\leftarrow$ *grade* $+ 5$.

(b) Repeat until A+

(c) If $y > 60$, print “many”.

(d) Sort the list somehow.

(b) and (d)

(b) is not *finite* or *unambiguous* — “A+” conditions are not defined.

(d) is not *effective* — “somehow” names no procedure.

The warning signs: *some*, *a few*, *until it looks right*, *etc.*

Decomposition

Any complex problem can be broken into simpler sub-problems.

Solve each in order; when all are solved,
the original problem is solved too.

the principle named after al-Khwārizmī, 9th century

Worked example — average of three numbers

Analyse

Input: N_1, N_2, N_3

Output: their average

Process: add the three, divide by 3

Algorithm

1. Start.
2. Get N_1, N_2, N_3.
3. Set $sum \leftarrow N_1 + N_2 + N_3$.
4. Set $avg \leftarrow sum/3$.
5. Print avg.
6. Stop.

Steps 3 and 4 could merge — but the order cannot change.

Pseudocode

Definition

An algorithm written in English-like phrases arranged to look like program statements — with no strict syntax rules.

```
START
INPUT a, b
SET sum = a + b
PRINT sum
END
```

No compiler means no single correct spelling:

`A = B + C` $A \leftarrow B + C$ `SET A = B + C`

Our convention: keywords in capitals, and indentation is the structure — there are no braces to fall back on.

Assignment is not equality

`SET x = x + 1` is not an equation

As mathematics it is false for every x. Read it **right to left**: take the current value of `x`, add one, store the result back into `x`.

YOUR TURN — what does each hold?

```
SET a = 20
SET b = a + 7
SET a = b * 2
```

Assignment is not equality

`SET x = x + 1` is not an equation

As mathematics it is false for every x. Read it **right to left**: take the current value of `x`, add one, store the result back into `x`.

YOUR TURN — what does each hold?

```
SET a = 20
SET b = a + 7
SET a = b * 2
```

$$a = 27, \qquad b = 54$$

Line 2 uses the *old* `a`. Line 3 does not touch `b`: assignment copies a value, it does not link two variables.

Selection: making a decision

```
INPUT points
IF points >= 95 THEN
   PRINT "A+"
ELSE
   PRINT "A0"
```

```
INPUT TEMP
IF TEMP > 27 THEN
   PRINT "Hot"
ELSE IF TEMP < 0 THEN
   PRINT "Cold"
ELSE
   PRINT "Nice"
```

A chain of `ELSE IF` tests in order and stops at the first true condition.

The order matters

Correct — strictest first

```
IF points >= 95 THEN
   PRINT "A+"
ELSE IF points >= 75 THEN
   PRINT "B+"
ELSE
   PRINT "B0"
```

Wrong — loosest first

```
IF points >= 75 THEN
   PRINT "B+"
ELSE IF points >= 95 THEN
   PRINT "A+"
ELSE
   PRINT "B0"
```

On the right, you will get B+ although you have enough points for A+.

Unreachable code

A branch that no input can reach

It is a logic error the compiler will never report. Your program is smaller than you think it is.

Iteration: repeating work

```
SET i = 1
WHILE i <= 5 DO
    PRINT i
    SET i = i + 1
```

Prints `1 2 3 4 5`.

Every loop has three parts

initialise `i = 1`

test `i <= 5`

update `i = i + 1`

Drop the update and the loop never ends.

`WHILE` tests *before* each pass — so it may run zero times.

Two patterns in almost every loop

Counter

`SET i = 0 ... SET i = i + 1`

Counts how many times something happened.

Accumulator

`SET total = 0 ... SET total = total + x`

Builds a running sum. It starts at 0 because 0 is the sum of no numbers.

`FOR i = 1 TO n` is the same loop with the bookkeeping folded into one line.

Worked example — sum of N numbers

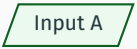
```
INPUT n
SET total = 0 // sum of nothing
SET i = 1
WHILE i <= n DO
   INPUT x
   SET total = total + x
   SET i = i + 1
PRINT total
```

Trace: $n = 3$; $4, 7, 2$

Pass	*total*	*i*
before	0	1
1	4	2
2	11	3
3	13	4

$4 \leq 3$ fails; prints **13**. ✓

Flowcharts: the same algorithm, drawn

Symbol	Name	Used for
	Oval	`Start` and `End`. Exactly one of each.
	Parallelogram	Input and output.
	Rectangle	A process: a calculation or an assignment.
	Diamond	A decision. Two arrows out, labelled `Yes` / `No`.

One algorithm, three notations

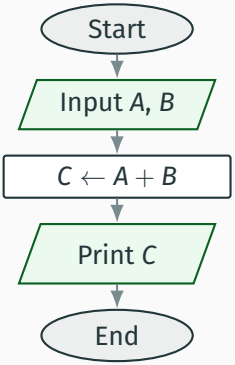
Natural language

1. Begin.
2. Input *A*, *B*.
3. Add them.
4. Display the result.
5. End.

Pseudocode

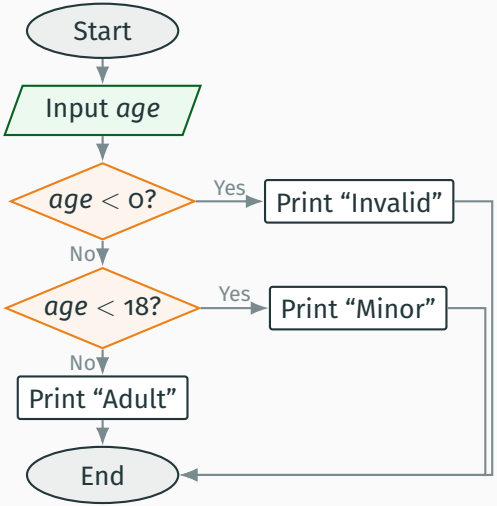
```
START
INPUT A, B
SET C = A + B
PRINT C
END
```

Flowchart



All three say exactly the same thing — and none of them is C++ yet.

A flowchart with two decisions

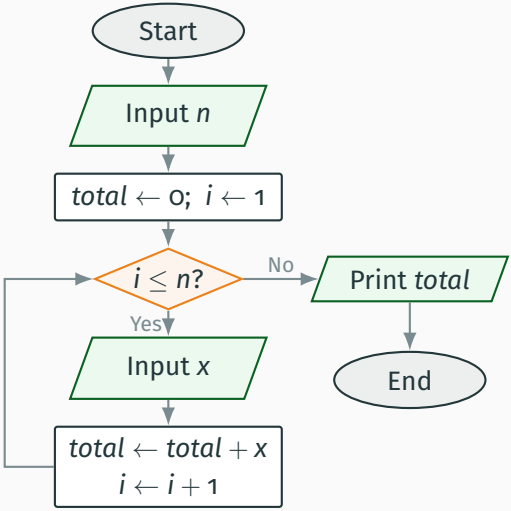


Read the diamonds in order

The second decision only ever sees a non-negative age — exactly like the second line of an `ELSE IF` chain.

Every diamond has two labelled arrows, and all paths reach the same `End`.

A flowchart with a loop



The arrow that makes it a loop

It runs *back up* into the decision — never into the initialisation.

Return above the initialisation and *total* resets to 0 on every pass.

Three flowchart faults that cost marks

1. A decision with one outgoing arrow, or branches not labelled `Yes` / `No`.
2. A loop whose return arrow goes back above the initialisation.
3. A chart with no `Start` or no `End` block.

None of these is about neat drawing — each one changes what the algorithm *means*.

Trace tables: testing before any code exists

```
SET a = 20
SET b = 30
IF a = 20 THEN
   SET b = 10
ELSE
   SET b = 5
IF b = 30 THEN
   SET a = 10
ELSE
   SET a = 30
```

Trace tables: testing before any code exists

```
SET a = 20
SET b = 30
IF a = 20 THEN
   SET b = 10
ELSE
   SET b = 5
IF b = 30 THEN
   SET a = 10
ELSE
   SET a = 30
```

Step	Action	*a*	*b*
1	`SET a = 20`	20	—
2	`SET b = 30`	20	30
3	`a = 20`? **true** → `b = 10`	20	10
4	`b = 30`? **false** → `a = 30`	30	10

Your turn — trace this loop

```
SET a = 1
SET b = 0
WHILE a <= 27 DO
   SET a = a * 3
   SET b = b + 1
PRINT a, b
```

Your turn — trace this loop

```
SET a = 1
SET b = 0
WHILE a <= 27 DO
   SET a = a * 3
   SET b = b + 1
PRINT a, b
```

Pass	$a <= 27$?	a	b
before	—	1	0
1	true	3	1
2	true	9	2
3	true	27	3
4	true	81	4
5	false	81	4

It prints **81** and **4** — a finishes *past* the limit.

Writing the condition column down is what catches an infinite loop.

The logic faults worth memorising

Fault	What to do instead
Ambiguous step	Replace *some* / *a few* with a number or a testable condition.
Infinite loop	The body must change a variable used in the test.
Uninitialised variable	Counters and accumulators are set *before* the loop.
Mis-ordered conditions	Test the strictest condition first.
Off-by-one	Decide explicitly: $\geq$ or $>$, $\leq n$ or $< n$.
Reset inside the loop	The return arrow goes to the *test*.

Before you accept any algorithm, trace it with a boundary value.

Break

C++: programs, variables, types

The smallest complete C++ program

```cpp
#include <iostream> // the library that gives us cin and cout
using namespace std; // so we can write cout, not std::cout

int main() { // execution starts here
   cout << "Hello, CS102!" << endl;
   return 0; // 0 = finished normally
}
```

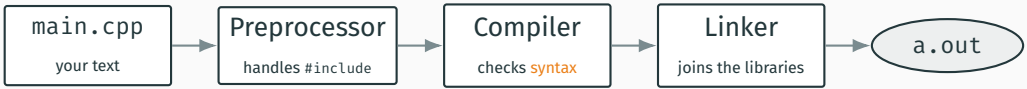
Two comment forms: `// to end of line` and `/* across several lines */`

Four things the compiler insists on

`#include`	Pastes a library's declarations in *before* compiling.
`int main()`	Every program has exactly one. The operating system calls it.
`{ }`	A block. Indentation means nothing here — braces do.
`;`	Ends every *statement* — not every line.

This is the difference between pseudocode and a language:
the machine needs the punctuation, not just the plan.

From your text file to a running program



syntax errors stop here

logic errors appear only *here*

The compiler is a grammar checker, not a proofreader.

Variables

Definition

A named region of memory whose contents can change while the program runs.

The type decides three things at once:

- how many **bytes** the box gets,
- how those bytes are **interpreted**,
- which **operations** are legal on it.

Declaring and initialising

```
int age; // declared, no value yet
int count = 0; // copy-initialised
double price = 27.11;
char grade = 'A'; // single quotes
bool passed = true;
```



Declaration creates the box; **initialisation** puts the first value in it. Do both on one line.

Naming

Rules — the compiler enforces these

- Letters, digits and _ only, never starting with a digit.
- Case sensitive: `count` $\neq$ `Count`.
- No keywords: `int`, `return`, `for`…

Conventions — good practice

- Say what it holds: `studentCount`, not `sc`.
- One idea per variable — do not reuse `temp`.
- Avoid `l`, `O`, `I` alone: they look like `1` and `0`.

```
int 2ndScore; int total-marks; int double; // all three: ERROR
```

Trap 1 — the uninitialised variable

```
int sum; // no value!
cout << sum << endl;
```

Output

```
32782
```

. . . or `15`, or `-1174405`, or anything else.

It is not zero — it is whatever was already in that memory

And the worst part: it often *appears* to work, because those bytes happened to be 0 — on your laptop, today.

Bits, storage and the primitive types

Bits and storage

The only formula you need

N bits hold 2^N patterns.

A hard ceiling: 8 bits distinguish 256 things.



one byte = $32 + 8 + 4 + 1 = 45$

The primitive types, and what they cost

Type	Bytes	Range	Use it for
`bool`	1	`false`, `true`	a yes/no flag
`char`	1	$-128 \ldots 127$	one character
`short`	2	$-32\,768 \ldots 32\,767$	small counters
`int`	4	$\pm 2.1 \times 10^9$	the default integer
`long long`	8	$\pm 9.2 \times 10^{18}$	IDs, money in tiyin
`float`	4	≈ 7 significant digits	rarely
`double`	8	≈ 15 significant digits	the default real number

Ask the compiler yourself

```
cout << sizeof(int) << endl; // 4
cout << sizeof(double) << endl; // 8
cout << sizeof(char) << endl; // 1
```

`signed` vs `unsigned`: the same bits, read differently

With N bits

`unsigned`: $0 \ldots 2^N - 1$

`signed`: $-2^{N-1} \ldots 2^{N-1} - 1$

You do not gain a bit — you move the window. The sign costs the top bit.

Type	Lowest	Highest
`char` (8)	−128	127
`unsigned char` (8)	0	255
`int` (32)	−2 147 483 648	2 147 483 647
`unsigned int` (32)	0	4 294 967 295

Trap 2 — the edge of the range

```cpp
unsigned char small = 255;
cout << (int)(small + 1) << endl;

unsigned int u = 0;
cout << u - 1 << endl;

int big = 2147483647; // INT_MAX
cout << big + 1 << endl;
```

Output

```
256
4294967295
-2147483648
```

(0 again if you store it back into the `char`)

Wrapping is defined; overflowing is not

Unsigned arithmetic wraps modulo 2^N — $0 - 1$ becomes the largest value there is. Signed overflow is *undefined behaviour*: the compiler may wrap, or assume it never happens and delete your `if`.

`char` is a small integer under the hood

```
char c = 'A';
cout << c << endl;
cout << c + 1 << endl;
cout << (char)(c + 1) << endl;
cout << (int)c << endl;
```

`char` is a small integer under the hood

```
char c = 'A';
cout << c << endl;
cout << c + 1 << endl;
cout << (char)(c + 1) << endl;
cout << (int)c << endl;
```

Output

```
A
66
B
65
```

Arithmetic promotes `char` to `int`

`'A'` is stored as **65**, so `c + 1` is the *integer* 66 — and `cout` prints an `int` as digits.

Useful: `digit - '0'` gives a digit's value; `'a' - 'A'` $= 32$. And `'A'` is not `"A"`.

Real numbers are approximations

```
double d = 0.1 + 0.2;
cout << d << endl;
cout << (d == 0.3) << endl;
cout << setprecision(20) << d << endl;
```

Real numbers are approximations

```
double d = 0.1 + 0.2;
cout << d << endl;
cout << (d == 0.3) << endl;
cout << setprecision(20) << d << endl;
```

Output

```
0.3
0
0.30000000000000004441
```

Never write `if (a == b)` for floating-point numbers

0.1 has no exact binary form, exactly as $1/3$ has no exact decimal one.

The first line only *looked* right: `cout` rounds to 6 significant digits by default.

Trap 3 — integer division

```
int a = 7, b = 2;
cout << a / b << endl;
cout << a % b << endl;
cout << (double)a / b << endl;

int sum = 7, n = 2;
double avg = sum / n;
cout << avg << endl;
```

Trap 3 — integer division

```cpp
int a = 7, b = 2;
cout << a / b << endl;
cout << a % b << endl;
cout << (double)a / b << endl;

int sum = 7, n = 2;
double avg = sum / n;
cout << avg << endl;
```

Output

```
3
1
3.5
3
```

`int / int` throws the fraction away — it does not round

`sum / n` is computed *entirely in* `int`; assigning to a `double` afterwards is too late.

Conversions: who wins when types are mixed

Widening — automatic, safe

`char` → `int` → `long long` → `double`

The narrower operand is promoted to the wider one.

Narrowing — silent loss

```
int n = 3.99; // n is 3
char c = 300; // overflows
```

Truncation is not rounding: `int n = -3.99;` gives -3.

Streams and formatted output

Output: cout and the << operator

```
int age = 20;
cout << "Age: " << age << " years" << endl;
cout << "A" << "B" << 1 << 2 << endl;
cout << "Line 1\nLine 2\n";
```

Output

```
Age:  20 years
AB12
Line 1
Line 2
```

Reading it

`cout` is the standard output stream; `<<` is the *insertion* operator — an arrow pointing into the stream. Each `<<` returns `cout` again, which is why they chain.

`cout` inserts no spaces of its own. Every space is one you typed.

`endl` or `\n`?

`endl` is `'\n'` *plus a flush* of the output buffer

Use it when the text must appear right now — a prompt just before `cin`, or a debug line before a crash.

`'\n'` just ends the line

Inside a loop printing thousands of lines it is noticeably faster, because the buffer is flushed once at the end.

Trap 4 — << binds tighter than you think

```
int a = 3, b = 5;
cout << "a < b: " << a < b << endl; // ERROR
cout << "a+b: " << a + b << endl; // fine
```

Line 2 parses as `(cout << "a < b: " << a) < b`

"Is this output stream less than 5?" — `<<` has *higher* precedence than comparison and bitwise operators. Arithmetic binds tighter than `<<`, which is why `a + b` needs no help.

Rule: anything other than arithmetic goes in parentheses — `<< (a < b) <<`.

Input: `cin` and the >> operator

```
int a, b;
cout << "Enter two numbers: ";
cin >> a >> b; // chains, like cout
cout << a + b << endl;
```

What >> does

1. Skips leading whitespace.

2. Reads as much as fits the type.

3. Stops at the first whitespace and leaves it in the buffer.

Because of step 1, `3  4` and `3` ENTER `4` behave identically.

Because of step 3, `cin >> name` reads one word — use `getline(cin, name)` for a full line.

Trap 5 — when `cin` fails, it stays failed

```
int age;
cout << "Age: ";
cin >> age; // user types: abc
cout << "You are " << age << endl;
```

Output

```
Age:  abc
You are 0
```

A failed extraction does three things

`age` becomes **0**, the failbit is set, and `abc` stays in the buffer — so every later `cin >>` fails instantly. That is the infinite prompt loop in your first menu program.

Formatting output: `#include <iomanip>`

Manipulator	What it does	Sticky?
`setw(n)`	Pads the *next* item to width *n*.	No — next item only
`setprecision(n)`	*n* significant digits — or *n* decimals once `fixed` is on.	Yes
`fixed`	Plain decimal notation, no exponent.	Yes
`setfill(c)`	Pads with `c` instead of a space.	Yes
`left` / `right`	Alignment inside the `setw` field.	Yes

Everything except `setw` stays in force until you change it.

`setw`: width for exactly one item

```
cout << setw(8) << "ab" << "cd" << endl;
cout << setfill('.') << setw(8) << 42 << endl;
```

Output

```
      abcd
......42
```

Only `"ab"` was padded — `setw` resets after one item. A stray `setfill('.')`, on the other hand, decorates the rest of your program.

`setprecision`: significant digits, or decimals

```
double pi = 3.14159265, big = 1234567.0;

cout << pi << endl;
cout << big << endl;
cout << setprecision(3) << pi << endl;
cout << fixed << setprecision(3) << pi << endl;
cout << fixed << setprecision(2) << 19.999 << endl;
```

`setprecision`: significant digits, or decimals

```cpp
double pi = 3.14159265, big = 1234567.0;

cout << pi << endl;
cout << big << endl;
cout << setprecision(3) << pi << endl;
cout << fixed << setprecision(3) << pi << endl;
cout << fixed << setprecision(2) << 19.999 << endl;
```

Output

```
3.14159
1.23457e+06
3.14
3.142
20.00
```

Default precision is 6 *significant* digits

Which is why `1234567.0` came out in scientific notation. With `fixed`, `setprecision` counts digits *after the point* — and it rounds. For money, once near the top of `main`: `cout << fixed << setprecision(2);`

Putting it together — a formatted table

```cpp
cout << left << setw(12) << "Item"
     << right << setw(6) << "Qty"
     << right << setw(10) << "Price" << endl;

cout << setfill('-') << setw(28) << "" << endl;
cout << setfill(' ') << fixed << setprecision(2);

cout << left << setw(12) << "Notebook"
     << right << setw(6) << 3
     << right << setw(10) << 12.5 << endl;
cout << left << setw(12) << "Pen"
     << right << setw(6) << 12
     << right << setw(10) << 1.75 << endl;
```

Output

```
Item           Qty      Price
----------------------------
Notebook         3      12.50
Pen             12       1.75
```

Three ideas: one `fixed` setting, a `setw` per column, and `setfill` to draw a rule.

Your turn — predict the output

```
int n = 5;
double score = 4;

cout << n / 2 << endl;
cout << n / 2.0 << endl;
cout << score / n << endl;
cout << 'B' - 'A' << endl;
cout << setw(4) << n << setw(4) << n << endl;
```

Your turn — predict the output

```cpp
int n = 5;
double score = 4;

cout << n / 2 << endl;
cout << n / 2.0 << endl;
cout << score / n << endl;
cout << 'B' - 'A' << endl;
cout << setw(4) << n << setw(4) << n << endl;
```

Output

```
2
2.5
0.8
1
1
   5   5
```

`score / n` works because one side is already a `double`.

The algorithm from Part I, now in C++

```
INPUT a
INPUT b
SET sum = a + b
SET avg = sum / 2
PRINT avg
```

```cpp
double a;
double b;
cout << "Enter a: ";
cin >> a;

cout << "Enter b: ";
cin >> b;

double sum = a + b;
double avg = sum / 2;

cout << fixed << setprecision(2) << "Average: " <<
      avg;
```

Wrap-up

Common errors summarized

	What you write	What actually happens
1	`int sum;` then `sum += x`	Starts from garbage. Works by luck.
2	`unsigned int u = 0; u - 1`	Wraps to 4 294 967 295.
4	`cout << c + 1` (`if char c`)	Prints 66, not B.
6	`double avg = sum / n;` (both `int`)	Integer division happens first.
7	`cin >> age;` with input `abc`	`age` is 0 and the stream stays failed.

Every one of these compiles. The compiler checks your grammar, never your meaning.

What to take away

Part I

- Plan before you type — the algorithm is the language-independent part.
- Pseudocode and flowcharts say the same thing two ways.
- A trace table with a boundary value finds the bug before the compiler sees it.

Part II

- A type is bits + interpretation.
- `unsigned` moves the window up; it does not add a bit, and it wraps.
- Streams chain — and fail silently.
- `fixed`, `setprecision`, `setw` formatting tools you may use widely this semester.

Questions?